

Take-Home Assignment: Loyalty Points System

Overview

You're building the backend for a retail loyalty program. Customers earn points on purchases, redeem them for rewards, and are assigned a tier based on their spending history.

Please write your solution in Java. You're free to pick any framework you'd reach for (Spring Boot, Javalin, Jersey, Dropwizard, or even just a plain `main` method) and whatever database strategy you think fits the problem — SQLite is perfectly fine, for example. We don't have a preference on Java version or libraries; use what you're most comfortable and productive with.

We suggest time-boxing this to 2–3 hours. A clean, well-reasoned partial solution is more valuable to us than an exhaustive one with no thought behind it. We'll use your submission as the starting point for a technical conversation in the onsite.

By the end, a customer should be able to make a purchase and earn points, check their current balance, and redeem those points for a reward — with the system automatically handling expiry so that old points can't be used after 12 months.

What to Build

Required

Complete these before moving on.

- **Earn points** — when a customer makes a purchase, they earn points (1 point per \$1 spent)
- **Redeem points** — a customer can redeem points for a reward from a catalog if they have sufficient balance
- **Check balance** — return a customer's current available point balance
- **Expiry** — points expire 12 months after they are earned and cannot be redeemed after that

Extended

Attempt these if you have time after the core requirements are solid.

- **Tier status** — customers have a tier (Silver / Gold / Platinum) based on their total spend in the rolling 12 months. Choose your own thresholds. Return the customer's current tier alongside their balance.
- **Refunds** — when a purchase is refunded, the points earned from it are clawed back

Stretch

Only attempt these if the above are complete.

- **Redemption order** — when points are redeemed, consume the points closest to expiry first
- **CLI** — build a CLI interface alongside or instead of a REST API. Example interactions:

```
1 $ points balance --user=alice
2 $ points earn --user=alice --purchase-id=order-123
3 $ points redeem --user=alice --reward=free-coffee
4
```

Deliverables

1. **Working code** with clear instructions to run it locally. Please submit this as a public GitHub repo and forward the link to your interviewer or HR contact.
2. **A design section in your README** that covers:
 - Your data model — the key tables or structures and why you designed them that way
 - At least one decision you made where there was a real trade-off
 - What you would change or add with more time
 - A short note on which AI tools you used and how you used them, if any. This is just context for our conversation in the onsite — there's no right or wrong answer here.

Ground Rules

- No authentication needed — assume user identity is passed in the request or command
- No frontend needed
- You may use any tools or resources you'd normally reach for as an engineer
- Write the code the way you would for production. We care about the things you'd want a teammate to inherit: clear naming, thoughtful error handling, sensible structure, and code that's easy to read and change. Depth matters more than breadth here — a smaller scope done well will land better than a larger scope done sloppily.
- You're welcome to use AI tools (Copilot, Claude, ChatGPT, Cursor, or anything else you'd normally use) however you'd use them in your day-to-day work. During the onsite we'll walk through your code together and talk about the decisions behind it, so make sure you're comfortable explaining and discussing any part of what you submit.
- Don't over-engineer. A focused, working solution with clear reasoning beats an ambitious one that's half-finished.

- If you make a simplifying assumption, note it in your README rather than over-specifying the problem yourself
-